

Assistente Inteligente com RAG para Dados e Documentos Espaciais

Documento Técnico

Global Solution
Generative AI & Advanced Nets

Grupo:

Júlia Aben-Athar - 566325

Edgar Lódula - 565260

Bruno Fernandes - 552574

FIAP | Junho de 2026

1. Introdução e Problema Escolhido

A nova economia espacial gera diariamente um volume crescente de documentos técnicos, relatórios científicos e boletins de monitoramento que precisam ser consultados de forma eficiente por pesquisadores, gestores e tomadores de decisão. O problema central abordado neste projeto é a dificuldade de extrair informações relevantes de um corpus heterogêneo de documentos espaciais (PDFs de relatórios da NASA/ESA, artigos científicos sobre clima e monitoramento ambiental, boletins de queimadas do INPE) sem a necessidade de ler cada documento integralmente.

A solução proposta é um Assistente Inteligente baseado em RAG (Retrieval-Augmented Generation) que permite ao usuário fazer perguntas em linguagem natural e receber respostas fundamentadas exclusivamente no conteúdo dos documentos carregados, com indicação das fontes consultadas.

1.1 Corpus de Documentos Utilizado

Os seguintes documentos foram utilizados na demonstração do sistema:

Documento	Fonte	Tema
INFOQUEIMA — Boletim Mensal (Jan/2024)	INPE	Monitoramento de focos de fogo via satélite
Exploring Earth's Interface with Space (2024)	ESA / NASA	Termosfera-ionosfera e missão de satélite
Global Warming Has Accelerated (2025)	Hansen et al.	Aceleração do aquecimento global
Global Warming — Supplementary Data	Hansen et al.	Dados suplementares de clima
NASA's Climate Adaptation Plan (2024)	NASA	Adaptação climática da agência
Satellite Constellation Optimization (arXiv)	Pesquisa acadêmica	Otimização de órbitas e agendamento de satélites

2. Arquitetura da Solução

A solução adota arquitetura RAG clássica em três camadas: ingestão de documentos, indexação vetorial e geração aumentada por recuperação. A interface é construída sobre Streamlit, permitindo interação via chat em navegador web.

2.1 Diagrama do Fluxo RAG

O fluxo completo de processamento segue as etapas abaixo:

- 1. INGESTÃO: Documentos PDF, TXT ou DOCX são carregados da pasta docs/ ou via upload.
- 2. CHUNKING: O texto extraído é dividido em trechos de 400 palavras com sobreposição de 60 palavras.
- 3. EMBEDDING: Cada trecho é transformado em vetor de 384 dimensões via API OpenAI (text-embedding-3-small).
- 4. INDEXAÇÃO: Os vetores são armazenados no índice FAISS com similaridade de cosenos.

- 5. CONSULTA: A pergunta do usuário é convertida para vetor pelo mesmo modelo de embedding.
- 6. RECUPERAÇÃO: Os top-k trechos mais similares à pergunta são recuperados do FAISS.
- 7. GERAÇÃO: Os trechos recuperados são injetados no contexto do GPT-4o-mini, que gera a resposta.
- 8. RESPOSTA: O assistente exibe a resposta e as fontes consultadas na interface Streamlit.

3. Ferramentas Utilizadas

Ferramenta	Função no Sistema
Python 3.13	Linguagem de implementação
Streamlit	Interface web interativa (chat)
OpenAI text-embedding-3-small	Geração de embeddings vetoriais via API
OpenAI GPT-4o-mini	Modelo generativo
FAISS (faiss-cpu)	Vector store para busca por similaridade
OpenAI Python SDK	Acesso às APIs de embeddings e geração
pypdf	Extração de texto de arquivos PDF
python-docx	Leitura de arquivos DOCX
python-dotenv	Gerenciamento de variáveis de ambiente

4. Modelos Utilizados

O projeto utiliza dois modelos da OpenAI, acessados via API oficial através do OpenAI Python SDK:

4.1 Modelo de Embedding

text-embedding-3-small — responsável por converter textos (chunks dos documentos e perguntas do usuário) em vetores de 384 dimensões. Esses vetores são armazenados no FAISS e utilizados para busca por similaridade semântica.

- Dimensão dos vetores: 384
- Normalização L2 aplicada antes da inserção e da consulta (equivale a cosine similarity)

4.2 Modelo Generativo

GPT-4o-mini — responsável pela etapa final do pipeline RAG: recebe o contexto dos trechos recuperados e a pergunta do usuário, e produz uma resposta em português fundamentada exclusivamente no conteúdo dos documentos. O modelo pode ser substituído via variável de ambiente OPENAI_MODEL no arquivo .env.

Parâmetros utilizados na chamada à API:

- Modelo: gpt-4o-mini (configurável via OPENAI_MODEL)
- max_tokens: 2048 tokens de saída
- System prompt: instrui o modelo a responder em português e citar as fontes dos documentos

5. Vector Store Utilizada

O projeto utiliza FAISS (Facebook AI Similarity Search), biblioteca open-source desenvolvida pela Meta AI para busca eficiente em espaços de alta dimensionalidade. A implementação é local (sem necessidade de servidor externo ou conta em serviço de nuvem).

5.1 Detalhes de Implementação

- Índice: IndexFlatIP (produto interno exato, sem aproximação)
- Dimensão: 384 (compatível com OpenAI text-embedding-3-small)
- Normalização L2 aplicada antes da inserção e da consulta — equivale a similaridade cosseno
- Busca: top-k trechos mais similares, k configurável (padrão: 5)
- Metadados: cada vetor é associado ao nome do arquivo de origem para citação

6. Fluxo RAG Implementado

O pipeline RAG está dividido em quatro módulos Python independentes, organizados no pacote rag/:

Módulo	Responsabilidade
rag/document_loader.py	Carrega PDF/TXT/DOCX e divide em chunks com sobreposição
rag/embeddings.py	Gera embeddings via API OpenAI (text-embedding-3-small)
rag/vector_store.py	Encapsula FAISS: add(), search(), clear(), total
rag/pipeline.py	Monta contexto e chama GPT-4o-mini via OpenAI SDK

A interface Streamlit (app.py) orquestra os módulos: na inicialização, carrega automaticamente todos os arquivos da pasta docs/; em cada pergunta, chama vector_store.search() seguido de pipeline.answer().

7. Limitações da Solução

- Dependência de API paga: tanto embeddings (text-embedding-3-small) quanto geração (GPT-4o-mini) exigem chave OpenAI e têm custo por token.
- Sem persistência do índice: ao reiniciar o app, os documentos são reprocessados do zero, consumindo créditos de embedding novamente.
- PDFs com layout complexo: tabelas, colunas duplas e imagens com texto não são extraídos corretamente pelo pypdf.
- Memória RAM: documentos muito grandes podem sobrecarregar a memória durante o processo de chunking e indexação.
- Sem histórico entre sessões: o chat e o índice são reiniciados a cada nova sessão do Streamlit.

8. Melhorias Futuras

- Persistência do índice FAISS em disco (faiss.write_index / read_index), eliminando o reprocessamento.
- Reranking com cross-encoder para melhorar a precisão dos trechos recuperados.
- Modelo de embedding multilíngue (ex: text-embedding-3-large ou modelos locais multilíngues) para melhor suporte ao português.
- Suporte a OCR para PDFs escaneados (via pytesseract ou pdfplumber).
- Interface para visualização dos chunks recuperados e seus scores de similaridade.
- Exportação do histórico de chat em PDF.
- Deploy em nuvem (Streamlit Cloud ou Hugging Face Spaces) para acesso remoto.

9. Conclusão

O sistema desenvolvido demonstra de forma funcional a arquitetura RAG aplicada ao domínio de dados espaciais. A combinação do modelo OpenAI text-embedding-3-small para geração de embeddings, FAISS para busca vetorial eficiente e GPT-4o-mini para geração de linguagem natural resulta em um assistente capaz de responder perguntas complexas sobre monitoramento de queimadas, clima espacial, adaptação climática da NASA e missões científicas ESA/NASA, sempre fundamentando as respostas no conteúdo dos documentos carregados.

A modularidade da implementação permite fácil extensão para novos tipos de documentos, outros modelos de linguagem e diferentes vector stores, tornando a solução uma base sólida para aplicações reais na nova economia espacial.